COMPUTER VISION · OBJECT DETECTION

Fridge Detector

A fine-tuned YOLOv8 model that looks inside your fridge, lists what it sees, and flags what's missing — built end-to-end, bug hunt included.

Mike Keror · github.com/mikekeror-rgb

THE IDEA

Point a camera at your fridge. Know what's inside — and what's not — in seconds.

No manual inventory. No guessing at the grocery store. A browser-based app that runs a custom-trained object detector entirely on-device.

The Pipeline

01
Open Images V7
Pulled via FiftyOne, balanced per class

→

02
Immich
Self-hosted canonical image store

→

03
YOLOv8 Fine-Tune
Ultralytics, GPU training on Colab

→

04
ONNX Export
Browser-ready inference format

→

05
Camera App
Live detection, fully on-device

A second loop runs on top of this: real photos get auto-labeled by the current model, corrected by hand, merged back in, and the model is retrained — repeatedly.

Nine Classes, Balanced On Purpose

Every class capped at the same target count during collection — the fix for a model that only ever learned to recognize the most common object in the room.

Class	Class	Class
Apple	Tomato	Bottle
Banana	Egg (Food)	Tin can
Orange	Milk	Cheese

THE BUG THAT BROKE EVERYTHING

Class indices got reordered across script versions — a box the model saw as "Bottle" one run was silently "Orange" the next. The model wasn't broken. The labels were lying to it.

0.006 → **0.519**

mAP50 before the fix

mAP50 after fixing the label mapping

Teaching It From Real Fridges

OpenImages taught the model shapes. Real, cluttered, badly-lit fridges taught it the truth. A human-in-the-loop cycle, run repeatedly:

Auto-label
Current model drafts boxes on new real photos

→

Human-correct
Fix wrong boxes, add missed ones, in Roboflow

→

Merge
Corrected examples folded into the training set

→

Retrain
Fine-tune continues from the last checkpoint

↻ Run across five model generations, each round targeting one confusion — tomato vs. orange, water bottle vs. tin can.

Before / After

TOMATOES

✕ labeled "Orange" (0.29–0.77)

↓

✓ labeled "Tomato" (0.66–0.86)

WATER BOTTLES

✕ labeled "Tin can" (0.51)

↓

✓ labeled "Bottle" (0.61–0.84)

Two rounds of targeted correction resolved two of the model's most consistent, repeatable confusions — clear proof the correction cycle actually works.

The App

A single-page browser app. Point your phone at the fridge, capture a frame, and the model runs entirely client-side — no server, no upload, no network round trip.

1 · Capture

getUserMedia opens the rear camera in-browser

2 · Detect

ONNX Runtime Web runs the model on-device

3 · List

Every item found, with a confidence score

4 · Alert

Flags any "essential" item that's missing

Tech Stack

Stage	Tools
Data collection	FiftyOne, Open Images V7, Immich (self-hosted API)
Labeling & correction	Roboflow, model-assisted auto-labeling
Training	Ultralytics YOLOv8n, Google Colab (T4 / A100 GPU)
Deployment	ONNX Runtime Web, vanilla JS, GitHub Pages

WHAT'S NEXT — PART 1

Region-Specific Groceries

Nine generic classes were enough to prove the pipeline. A real product needs to recognize what's actually in the fridge — and that changes by country, kitchen, and culture.

Local staples

Sukuma wiki, ugali flour, chapati, tilapia — trained on the food people actually buy

Local packaging

Region-specific brands and bottle or box shapes the model has never seen

Crowd-sourced correction

The same auto-label, correct, retrain loop — run per region

WHAT'S NEXT — PART 2

Built Into the Fridge Itself

The same model, running where it belongs: inside the fridge's own internal camera, checked from anywhere.

Fridge camera
Built-in camera (Samsung Family Hub, LG InstaView) captures the shelves

→

Manufacturer cloud
Model hosted in the fridge maker's own cloud, runs automatically

→

Companion app
Check contents from the grocery store, any device, anywhere

No manufacturer lock-in for the model itself — the same fine-tuning pipeline built here could plug into any smart-fridge platform's camera feed.

Thank You

From a raw dataset pull to a live, camera-driven detector — and every bug in between.

github.com/mikekeror-rgb